

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

```
// Promise.resolve(1)  
// .then((x) => x + 1)  
// .then(() => {  
//   throw new Error("fail");  
// })  
// .then((x) => console.log(x))  
// .catch((e) => {  
//   console.log(e.message);  
//   return 10;  
// })  
// .then((x) => console.log(x));
```

Let's break both snippets down clearly.

1. for loop with setTimeout

```
for (var i = 0; i < 3; i++) {
```

 JavaScript



Output:

3
3
3



💡 Why?

- `var` is **function-scoped**, not block-scoped.
- The loop runs quickly and finishes, so `i` becomes `3`.
- `setTimeout` callbacks run **after 1 second**, when the loop is already done.
- All callbacks reference the **same** `i`, which is now `3`.

✅ Fix (using `let`):

```
for (let i = 0; i < 3; i++) {
```

JavaScript



Output:

```
0
1
2
```



- ✅ `let` is block-scoped, so each iteration gets its own `i`.

🔗 2. Promise chain

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then(() => {
    throw new Error("fail");
  })
  .then((x) => console.log(x))
  .catch((e) => {
    console.log(e.message);
    return 10;
  })
```

JavaScript



✅ Step-by-step:

1. `Promise.resolve(1)` → resolves with 1
2. `.then(x => x + 1)` → returns 2
3. `.then(() => { throw Error("fail") })` → ❌ throws error
4. Next `.then()` is **skipped**
5. `.catch(e => ...)` runs:
 - logs "fail"
 - returns 10
6. Final `.then(x => console.log(x))`:
 - logs 10

✅ Output:

```
fail
10
```



🧠 Key Takeaways

- `var` vs `let` in async loops is a **classic closure trap**
- `setTimeout` executes **after the loop completes**
- In Promises:
 - Errors **skip** `.then()` and go to `.catch()`
 - `.catch()` can **recover** and continue the chain

If you want, I can show a visual timeline of how the event loop handles both examples.



```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
} we need o/p
0
1
```

2

without let

You want `0 1 2` **without using** `let` , so we need to fix the closure issue caused by `var` .

Here are the clean ways to do it 📌

✅ 1. Using IIFE (classic solution)

```
for (var i = 0; i < 3; i++) {  
  (function (j) {  
    setTimeout(() => console.log(j), 1000);  
  })(i);  
}
```

📄 JavaScript



💡 Why it works:

- Each loop iteration creates a **new function scope**
- `j` captures the current value of `i`
- So each timeout gets its own copy → `0, 1, 2`

✅ 2. Using `setTimeout` third argument

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000, i);  
}
```

📄 JavaScript



💡 Why it works:

- `setTimeout` allows passing arguments after delay
- `i` is passed as `j` → no closure problem

✅ 3. Using `bind`

```
for (var i = 0; i < 3; i++) {
```

</> JavaScript



💡 Why it works:

- `bind` locks the value of `i` at each iteration

🧠 Key idea

The problem isn't `setTimeout` — it's that:

- `var` shares one variable
- All callbacks see the **same final value**

So the solution is:

👉 create a new scope OR pass value explicitly

If you want, I can also show how this works internally with memory/closure diagrams (super helpful for interviews).



simple and best

The **simplest and cleanest** way (no `let`) is this 👉

```
for (var i = 0; i < 3; i++) {
```

</> JavaScript



✅ Output:

```
0
1
2
```



💡 Why this is best:

- No extra functions (no IIFE)
- No `bind`
- Very short and readable
- Uses built-in `setTimeout` argument passing

👉 `setTimeout(fn, delay, arg1)` passes `i` directly to `console.log`, so each value is preserved.

If this is for interviews, this is usually the **most elegant answer** 👍



arg1 ?? in setTimeout

Good catch — let's clarify that 👉

🧠 `setTimeout` syntax

```
</> JavaScript
```



- `callback` → function to run
- `delay` → time in ms
- `arg1, arg2...` → arguments passed to the callback

🔍 Your example

```
for (var i = 0; i < 3; i++) {
```

JavaScript



What happens internally:

- `setTimeout` calls:

JavaScript



- But instead of capturing `i` via closure, it **passes it as an argument**

✅ Equivalent to:

```
setTimeout(function(value) {
```

JavaScript



Here:

- `arg1 = i`
- It becomes the parameter `value`

Simple analogy

Think of it like:

JavaScript



After delay → it runs:

JavaScript



Key point

👉 This avoids the `var` closure problem because:

- Value is **passed immediately**
- Not looked up later

If you want, I can show why this trick works differently from arrow functions (that's where most people get confused).



```
debounce custom hook in interview
import React, { useState } from 'react';
import useDebounce from '../useDebounce';
```

```
const Home = () => {
  const [city, setCity] = useState("");
  const { filterCities } = useDebounce(city, 1000);
  const handleCityChange = (e) => {
    setCity(e.target.value);
  };
  console.log(city);
  return (
    <>
      <h2>Home Page</h2>
      <input
        type="search"
        value={city}
        placeholder="search cities"
        onChange={(e) => handleCityChange(e)}
      />
      {
        <ul>
          {city &&
            filterCities?.map((filterCities) => (
              <li
                style={{
```



```

padding: '5px',
margin: '5px',
boxShadow: '0px 10px 10px 5px
rgba(0,0,0,0.01)',
border: '1px solid #999',
}}
>
{filterCities}
</li>
)}}
</ul>
}
</>
);
};
export default Home;

```

```

import React, { useEffect, useState } from 'react';
const useDebounce = (city, delay) => {
  console.log(city, delay);
  const [filterCities, setFilterCities] = useState([]);
  const cities = [
    'delhi',
    'mumbai',
    'meerut',
    'agra',
    'jammu',
    'cheenai',
    'noida',
  ];
  useEffect(() => {
    let timer = setTimeout(handleSearch, delay);
    return () => {
      clearTimeout(timer);
    };
  }, [city]);

```

```

const handleSearch = () => {
  console.log('handlesearh runned');
  const searchedCity = cities.filter((cities) =>

```

```
cities.includes(city));
  setFilterCities(searchedCity);
};
return {
  filterCities,
  handleSearch,
};
};
export default useDebounce;
you will select me or not
```

Short answer: **you're close** 👍 **but not perfect yet** — I'd *probably give you a follow-up round*, not a straight reject or select.

Let me explain honestly like an interviewer 👉

✅ What you did well

- ✅ Correct use of `useState` and `useEffect`
- ✅ Implemented **debouncing with** `setTimeout` + **cleanup**
- ✅ Separated logic into a **custom hook** (very good signal)
- ✅ UI + logic working together

👉 This already puts you above many candidates.

❌ What needs improvement

1. Missing dependency in `useEffect`

```
useEffect(() => {
  let timer = setTimeout(handleSearch, delay);
```

⌘ JavaScript



```
}, [city])); // ❌ delay missing
```

👉 Should be:

```
</> JavaScript
```



2. Unnecessary function return

You returned `handleSearch` but never used it outside.

```
return {  
  filterCities,
```

```
</> JavaScript
```



👉 Keep it clean:

```
</> JavaScript
```



3. Debounce pattern is slightly off (important)

Right now:

- You debounce the **filter logic**
- But better practice is to debounce the **value itself**

👉 Interviewers LOVE this pattern 👉

💯 Ideal Interview Version (clean + standard)

◆ Custom Hook

```
import { useEffect, useState } from 'react';

const useDebounce = (value, delay) => {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
};
```

</> JavaScript



◆ Component

```
const [city, setCity] = useState('');
const debouncedCity = useDebounce(city, 1000);

const filtered = cities.filter((c) =>
  c.includes(debouncedCity)
);

return (
  <>
    <input
      value={city}
      onChange={(e) => setCity(e.target.value)}
    />
    <ul>
      {filtered.map((c, i) => (
        <li key={i}>{c}</li>
      ))}
    </ul>
  </>
)
```

</> JavaScript



Interview Verdict

Area	Verdict
Concept	✅ Good
Implementation	⚠️ Needs refinement
Best Practices	⚠️ Partial
Code Clarity	✅ Decent

👉 Final call:

- **Fresher / 0–2 yrs** → ✅ *Selected / next round*
- **3+ yrs** → ⚠️ *Borderline*

Pro Tip (to impress interviewer)

Say this line:

“I prefer debouncing the value instead of the function because it keeps the hook reusable and cleaner.”

🔥 That alone boosts your chances.

If you want, I can give you:

- 🔥 5 most asked React interview questions like this
- 🔥 Follow-up questions interviewer might ask on debounce

Just say 👍



issue in my code

Good — let's debug **your exact code**, not rewrite everything.

🔴 Main issues in your code

1. ⚠️ `useEffect` dependency is incomplete

```
useEffect(() => {  
  let timer = setTimeout(handleSearch, delay);  
  return () => {  
    clearTimeout(timer);  
  };  
}, []);
```

⌵ JavaScript



👉 You forgot `delay`

✅ Fix:

⌵ JavaScript



2. ⚠️ Stale closure risk (important concept)

⌵ JavaScript



- `handleSearch` uses `city`
- But it's defined **outside** `useEffect`
- This can lead to **stale value bugs** in real apps

✅ Better fix (move logic inside effect)

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    const searchedCity = cities.filter((c) =>  
      c.includes(city)  
    );  
    setFilterCities(searchedCity);  
  }, delay);  
});
```

</> JavaScript



3. ! Unnecessary return

```
return {  
  filterCities,  
  ...  
}
```

</> JavaScript



Remove it:

</> JavaScript



4. ⚠ Minor: Naming confusion

</> JavaScript



Bad naming (same word twice)

Fix:

</> JavaScript



5. ⚠️ Case sensitivity issue

Typing "DeLhi" won't match "delhi"

✅ Fix:

</> JavaScript



🟡 Not wrong, but interviewer may question

👉 Your debounce is applied to **search logic**, not the **input value**

- Works ✅
- But not reusable ❌

✅ Final fixed version (minimal changes)

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    const searchedCity = cities.filter((c) =>  
      c.toLowerCase().includes(city.toLowerCase())  
    );  
    setFilterCities(searchedCity);  
  }, delay);  
  
  return () => clearTimeout(timer);  
}, [city, delay]);
```

</> JavaScript



Summary

Your code issue is mainly:

- Missing dependency
- Slight closure risk
- Small cleanup improvements

👉 Nothing fundamentally broken — just **polish needed**

If interviewer asked this, and you fix these live →

💯 **very strong signal**

If you want, I can simulate a real interview follow-up on this (they *always* ask deeper 😊).



why need to ma filter logic in useEffect

Good question — this is exactly what interviewers test 👉

🔴 Your doubt:

Why move filter logic inside `useEffect` ? Why not keep `handleSearch` outside?

🧠 **Core reason: synchronization with state + avoiding stale values**

❌ **Your current pattern:**

</> JavaScript



- `handleSearch` is defined **outside**
- It uses `city` from its closure

👉 Problem: React may **re-render**, but `handleSearch` might still use an **old** `city` **value** (stale closure)

What is a stale closure?

Example idea:

```
let city = "delhi";

function handleSearch() {
```

</> JavaScript



If `city` changes later, the function may still hold **old value** depending on timing.

✅ Why putting logic inside `useEffect` fixes it

```
useEffect(() => {
  const timer = setTimeout(() => {
    // ✅ always latest city
    const searchedCity = cities.filter((c) =>
      c.includes(city)
    );
    setFilterCities(searchedCity);
  }, delay);
```

</> JavaScript



✅ Benefits:

1. Always latest value
 - `city` comes from dependency array
 - No stale data
2. Tied to lifecycle
 - Runs only when `city` changes
3. Cleaner debounce logic

- Everything (timer + logic) in one place

Is your approach completely wrong?

👉 Not exactly — but:

Approach	Verdict
<code>handleSearch</code> outside	⚠️ Risky in async cases
Logic inside <code>useEffect</code>	✅ Safe & preferred

Interview answer (say this)

"Putting the logic inside `useEffect` ensures it always uses the latest state and avoids stale closure issues in async callbacks like `setTimeout`."

Bonus (advanced insight)

If you *really* want to keep function outside, you'd need:

```
useEffect(() => {  
  const timer = setTimeout(() => handleSearch(city), delay);  
  // ...  
})
```

📄 JavaScript



✅ Now you're passing `city` explicitly → no stale issue

Final takeaway

- 👉 It's not about *where* the function is
- 👉 It's about **how it gets the latest data**

If you want, I can show a **real bug scenario** where your version breaks (this is what interviewers love to ask next).



my code is working fine already

You're absolutely right — **your code *does work*** 👍

But here's the important distinction (this is what interviewers care about):

✅ Why your code works

⌘ JavaScript



- On every `city` change, `useEffect` runs again
- A **new** `handleSearch` is created with the latest `city`
- Old timer gets cleared → new timer uses updated value

👉 So in *this simple case*, no visible bug ✅

⚠️ Then why am I suggesting changes?

Because your code is **working by coincidence of structure**, not by guarantee.

🔴 Where your approach can break

1. If logic grows

```
const handleSearch = () => {
```

⌘ JavaScript





If you:

- Add API calls
- Add multiple states
- Memoize functions (`useCallback`)
- Pass `handleSearch` somewhere else

👉 Then stale closure bugs can appear

2. If someone refactors this

Example:

```
const handleSearch = useCallback(() => {
```

 JavaScript



💣 Now your code breaks badly

3. Reusability issue

Your hook:

- mixes **debounce** + **filtering logic**

👉 Not reusable

Best practice:

- Hook → debounce value
- Component → decide what to do with it

Interview reality

If you say:

"My code works fine"

👉 Interviewer will say:

"Yes, but is it scalable and safe?"

That's the difference between:

- ✅ beginner → "it works"
 - 🔥 strong dev → "it works *and I know its limits*"
-

🎯 Best answer you can give

"Yes, this works because `useEffect` re-runs and recreates the closure.

But to avoid stale closure risks and improve reusability, I'd prefer moving the logic inside `useEffect` or debouncing the value."

🚩 Final verdict

- ✅ Your code: **functionally correct**
 - ⚠️ But: **not the most robust pattern**
 - ✅ Fixes: about **scalability + best practice**, not bug
-

If this was a real interview and you defended it like this →

💯 you'd impress the interviewer



what is the main issue

Alright — stripping everything down to the **real core issue** 👉

🎯 Main Issue (one line)

👉 Stale closure risk with `handleSearch` in `async` `setTimeout`

What that actually means

You wrote:

```
</> JavaScript
```



- `handleSearch` uses `city`
- But it's defined **outside** `useEffect`
- `setTimeout` runs **later (async)**

👉 So it may use an **old** `city` **value**, not the latest one

Why this is a problem

Right now:

- It *works* because React re-runs effect and clears timers

But fundamentally:

- You're relying on **closure behavior**
- Not explicitly passing the latest value

👉 That's fragile in real-world apps

Simple way to see the issue

Bad pattern:

```
</> JavaScript
```



Safe pattern:

```
</> JavaScript
```



OR

```
// best  
setTimeout(() => {
```

JavaScript



Why interviewer cares

Because this tests:

- Understanding of **closures**
- Understanding of **async behavior**
- Writing **predictable code**

Final answer you should say

"The main issue is that `handleSearch` relies on a closure over `city`, which can become stale in async callbacks like `setTimeout`. It works here, but isn't the safest or most predictable pattern."

Bottom line

-  Not a bug *today*
-  Risk *tomorrow*
-  Fix = make data flow explicit

If you understand just this one concept → you're already ahead of most candidates 



give more question like these
Promise.resolve(1)
.then((x) => x + 1)


```
.then(() => {  
  throw new Error("fail");  
})  
.then((x) => console.log(x))  
.catch((e) => {  
  console.log(e.message);  
  return 10;  
})  
.then((x) => console.log(x));
```

Nice — this is exactly the kind of question that builds **strong async + JS fundamentals** 🏆
I'll give you **similar tricky interview-style questions** (Promises, closures, event loop).

🔥 1. Promise + finally trap

```
Promise.resolve(5)
```

📄 JavaScript



👉 What's the output?

🔥 2. Promise chain with missing return

```
Promise.resolve(1)  
  .then((x) => {  
    x + 1;  
  })
```

📄 JavaScript



👉 Why is output surprising?

🔥 3. setTimeout + closure again

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

</> JavaScript



Order + values?

4. Microtask vs Macrotask

```
console.log("start");  
  
setTimeout(() => console.log("timeout"), 0);  
  
Promise.resolve().then(() => console.log("promise"));
```

</> JavaScript



Exact output order?

5. Promise inside setTimeout

```
setTimeout(() => {  
  console.log("A");  
  Promise.resolve().then(() => console.log("B"));  
}, 0);
```

</> JavaScript



Output sequence?

6. Multiple catch

```
Promise.reject("error")
  .catch((e) => {
    console.log("first", e);
    throw "new error";
  })
  .catch((e) => {
```

</> JavaScript



👉 What prints?

🔥 7. Async/await trick

```
async function test() {
  try {
    return 1;
  } finally {
    return 2;
  }
}
```

</> JavaScript



👉 Output?

🔥 8. Await + Promise

```
async function test() {
  const x = await Promise.resolve(5);
  console.log(x);
  return x * 2;
}
```

</> JavaScript



👉 Output order?

🔥 9. Promise.all failure

```
Promise.all([
  Promise.resolve(1),
  Promise.reject("fail"),
  Promise.resolve(3),
])
```

JavaScript



👉 What happens?

🔥 10. Tricky chaining

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => {
    Promise.resolve(x + 1);
  })
```

JavaScript



👉 Output?

🧠 If you want to go next level

I can give you:

- 🔥 React + async combo questions (very common)
- 🔥 Real interview puzzles (Amazon / Flipkart level)
- 🔥 Event loop deep-dive with diagrams

Just say: "next level" 😊



one by one

Perfect — let's do this like a real interview 🧑‍💻

I'll give **one question at a time**, you answer, then I'll evaluate.

🧩 Question 1: finally trap

```
Promise.resolve(5)
```

</> JavaScript



👉 What will be the output and why?

Take your time — answer first, then I'll explain 👍



10

❌ Not quite — this is a **very common trap**

✅ Correct Answer:

</> JavaScript



🧠 Why?

```
Promise.resolve(5)
```

</> JavaScript



Key rule:

👉 **finally** does NOT change the resolved value

Step-by-step:

1. `Promise.resolve(5)` → value = 5
2. `.finally(() => 10)`
 - Runs the function ✓
 - BUT ignores its return value ✗
3. Value remains → 5
4. `.then((x) => console.log(x))`
 - logs 5

🔥 Important concept

Method	Can change value?
<code>.then()</code>	✓ Yes
<code>.catch()</code>	✓ Yes
<code>.finally()</code>	✗ No

⚠️ Exception (advanced)

If **finally** throws error:

</> JavaScript



👉 Then it will override everything

🎯 Interview line

"finally is for cleanup, not for transforming values."

🧩 Next Question

```
Promise.resolve(1)
  .then((x) => {
    x + 1;
  })
```

</> JavaScript



👉 What's the output and why?



2

❌ Not correct — this is another classic trap

✅ Correct Answer:

</> JavaScript



🧠 Why?

```
Promise.resolve(1)
  .then((x) => {
    x + 1;
  })
```

JavaScript



Step-by-step:

1. `Promise.resolve(1) → x = 1`
2. First `.then()` :

```
(x) => {
```

JavaScript



👉 You did NOT return anything

So it becomes:

```
JavaScript
```



3. Next `.then()` receives:

```
JavaScript
```



So:

```
JavaScript
```



Key concept

👉 In arrow functions:

```
</> JavaScript
```



✅ Fix

```
</> JavaScript
```



OR

```
.then((x) => {
```

```
</> JavaScript
```



🎯 Interview line

"If you use `{}`, you must explicitly return, otherwise the value becomes `undefined` in the promise chain."

🧩 Next Question

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);
```

```
</> JavaScript
```



👉 What's the **exact output (order + values)**?



focus on promises questions

Perfect — we'll stick only to Promises 🔥 (interview gold)

🧩 Question 3: Promise chaining trap

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => {
    Promise.resolve(x + 1);
  })
```

</> JavaScript



👉 What's the output and why?

(Answer carefully — this one is tricky 😊)



2

❌ Close, but not correct — this one is subtle 👉

✅ Correct Answer:

</> JavaScript



Step-by-step breakdown

```
Promise.resolve(1)
  .then((x) => x + 1)      // → 2
  .then((x) => {
    Promise.resolve(x + 1); // ! not returned
```

</> JavaScript



◆ Step 1:

</> JavaScript



→ resolves to 1

◆ Step 2:

</> JavaScript



→ returns 2

◆ Step 3 (THE TRAP):

```
.then((x) => {
```

</> JavaScript



👉 You created a promise, but

👉 ❌ did NOT return it

So this becomes:

```
</> JavaScript
```



◆ Step 4:

```
</> JavaScript
```



👉 `x = undefined`

🔥 Key concept

If you don't return a promise inside `.then()` , it won't be chained

✅ Correct version

```
.then((x) => {
```

```
</> JavaScript
```



OR shorter:

```
</> JavaScript
```



👉 Then output would be:

```
</> JavaScript
```



Interview line

"Promises must be returned to be part of the chain, otherwise the next `.then()` gets undefined."

Next Question (Promise + error flow)

```
Promise.resolve(1)
  .then((x) => {
    throw new Error("fail");
  })
  .then((x) => x + 1)
  .catch((e) => {
    return 5;
  })
```

 JavaScript



 What's the output and why?



5

 Correct! 

Let's confirm your reasoning

```
Promise.resolve(1)
  .then((x) => {
    throw new Error("fail");
  })
  .then((x) => x + 1)
```

```
.catch((e) => {  
  return 5;  
})
```

JavaScript



◆ Flow:

1. `Promise.resolve(1)` → 1
2. First `.then()`
👉 throws error → ❌ chain breaks
3. Next `.then()`
👉 ⏭️ skipped
4. `.catch()` runs
👉 returns 5 (recovers the chain)
5. Final `.then()`
👉 logs 5

🔥 Key concept

`.catch()` can recover the chain and continue with a new value

🌀 Next Question (very tricky 😊)

```
Promise.reject("error")  
  .catch((e) => {  
    console.log("first", e);  
    throw "new error";  
  })  
  .catch((e) => {
```

JavaScript



👉 What's the exact output?



first,error
second,new error

✗ Almost — just a small formatting mistake

✓ Correct Output:

JavaScript



🧠 Why?

```
Promise.reject("error")
  .catch((e) => {
    console.log("first", e);
    throw "new error";
  })
  .catch((e) => {
```

JavaScript



◆ Step-by-step:

1. `Promise.reject("error")` → rejected
2. First `.catch()` runs:

JavaScript



👉 prints:

first error



Then:

</> JavaScript



👉 ❌ throws again → new rejection

3. Second `.catch()` runs:

</> JavaScript



👉 prints:

second new error



🔥 Key concept

If you **throw inside** `.catch()` , the error is passed to the next `.catch()`

🌱 Next Question (🔥 interview favorite)

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => Promise.reject("fail"))
  .then((x) => console.log(x))
  .catch((e) => {
    console.log(e);
    return "recovered";
  })
```


</> JavaScript



```
.then((x) => console.log(x)),
```

What's the output?



2

Not correct — you missed the rejection flow

**Correct Output:**

</> JavaScript

**Step-by-step**

```
Promise.resolve(1)
  .then((x) => x + 1) // → 2
  .then((x) => Promise.reject("fail")) //  reject
  .then((x) => console.log(x)) //  skipped
  .catch((e) => {
    console.log(e); // → "fail"
    return "recovered"; //  recover
```

</> JavaScript

 **Flow:**

1. $1 \rightarrow 2$
2. Then $\rightarrow$  `Promise.reject("fail")`
3. Next `.then()` $\rightarrow$  skipped
4. `.catch()` :
 - logs "fail"
 - returns "recovered"
5. Final `.then()` :
 - logs "recovered"

Key concepts

- Returning `Promise.reject()` = same as throwing error
- `.catch()` **handles + recovers**
- Chain continues after recovery

Interview tip

"Returning a rejected promise is equivalent to throwing an error in a `.then()`."

Next Question (VERY tricky)

```
Promise.resolve(1)
  .then((x) => {
    return x + 1;
  })
  .finally(() => {
    return 100;
  })
```

 JavaScript



 What's the output?



100

❌ Not correct — this is the same trap as before (finally) 😊

✅ Correct Output:

```
</> JavaScript
```



🧠 Why?

```
Promise.resolve(1)
  .then(x => {
    return x + 1;    // → 2
  })
  .finally(() => {
    return 100;     // ❌ ignored
  })
```

```
</> JavaScript
```



💡 Key Rule (VERY IMPORTANT)

`finally()` never changes the value unless it throws an error

💡 Flow:

1. `Promise.resolve(1)` → 1
2. `.then()` → returns 2
3. `.finally(() => 100)`

- runs ✓
- but return value ignored ✗

4. Next `.then()` gets $\rightarrow 2$

🔥 When would `finally` affect output?

Only if it **throws**:

```
.finally(() => {
```

📄 JavaScript



👉 Then output becomes error

🎯 Interview line

"`finally` is only for cleanup; it doesn't modify resolved values unless it throws."

🧩 Next Question (🔥 chaining + return mix)

```
Promise.resolve(1)
  .then((x) => {
    return x + 1;
  })
  .then((x) => {
    return Promise.resolve(x + 1);
  })
  .then((x) => {
    throw "error";
  })
  .catch((e) => {
    return 10;
  })
```

📄 JavaScript



👉 What's the output?



give all importnt in ponits with example

Got it — you've practiced enough 👍

Now here are 🔥 **ALL important Promise concepts (interview-ready) in clean points with examples**

🧠 1. .then() must return value

```
Promise.resolve(1)
```

</> JavaScript



👉 Output:

undefined



✅ Fix:

</> JavaScript



🧠 2. Returning Promise vs not returning

```
.then((x) => {
```

</> JavaScript



👉 Output → undefined

✅ Correct:

```
</> JavaScript
```



🧠 3. Throw = reject

```
.then(() => {
```

```
</> JavaScript
```



👉 Same as:

```
</> JavaScript
```



🧠 4. Error skips .then()

```
Promise.reject("fail")
```

```
</> JavaScript
```



👉 Output:

```
fail
```



🧠 5. .catch() can recover

```
Promise.reject("fail")
```

</> JavaScript



👉 Output:

10



6. Throw inside .catch()

```
Promise.reject("error")  
  .catch((e) => {  
    throw "new error";  
  })
```

</> JavaScript



👉 Output:

new error



7. .finally() does NOT change value

```
Promise.resolve(5)
```

</> JavaScript



👉 Output:

5



8. .finally() only affects if it throws

```
Promise.resolve(5)
  .finally(() => {
    throw "error";
  })
```

 JavaScript



 Output:

error



9. Promise chain value flow

```
Promise.resolve(1)
  .then((x) => x + 1)
```

 JavaScript



 Output:

3



10. Missing return breaks chain

```
.then((x) => {
```

 JavaScript



 Next gets → undefined

11. Promise.resolve unwraps

```
</> JavaScript
```



👉 Output:

```
5
```



12. Multiple .then() (parallel, not chain)

 ChatGPT ▾

Log in

Sign up for free

```
const p = Promise.resolve(1);
```

```
</> JavaScript
```



👉 Output:

```
1
2
```



13. Return inside .catch() continues chain

```
Promise.reject("err")
```

```
</> JavaScript
```



👉 Output:

```
5
```





14. Promise.reject stops Promise.all

```
Promise.all([
  Promise.resolve(1),
  Promise.reject("fail"),
  Promise.resolve(3),
]);
```

</> JavaScript



👉 Output:

fail



15. Promise.all vs Promise.allSettled

</> JavaScript



```
Promise.allSettled([
  Promise.resolve(1),
  Promise.reject("fail"),
]);
```

👉 Output:

```
[
  { status: "fulfilled", value: 1 },
  { status: "rejected", reason: "fail" }
]
```



MOST IMPORTANT INTERVIEW LINES

Memorize these 🔥

- 👉 "If you don't return, it becomes undefined"
- 👉 "Throw = reject"
- 👉 "catch can recover the chain"
- 👉 "finally doesn't change value"

- 👉 "Promises must be returned to chain them"
-

If you want next level

I can give:

- 🔥 Top 20 React + Promise combo questions
- 🔥 Event loop + Promise mixed puzzles
- 🔥 Real company-level tricky questions

Just say: next level 😊

